In this lab, you will implement Gaussian Elimination and test your implementation by solving a linear system. Sample runs of the forward and backward steps are available in the Lab 6 Sample Run document on Quercus – we recommend reading through the sample runs to understand the steps of Gaussian elimination before starting to write any code. The TAs are here to help you. If you are stuck, ask for help!

Recall that in Python, you can use a list of lists to store a matrix, with each inner list representing a row. For example, you can store the matrix

$$\begin{bmatrix} 5 & 6 & 7 \\ 0 & -3 & 5 \end{bmatrix}$$

by storing each row as a list: `M = [[5, 6, 7], [0, -3, 5]]`. For ease of reading, since Python allows for line breaks inside brackets, you can write it as follows:

```
M = [[5,  6, 7],
     [0, -3, 5]]
```

You can use `M[1]` to access the second row of `M` (i.e., `[0, -3, 5]`), and you can use `M[1][2]` to access the third entry in the second row (i.e., `5`).

You will first implement some helper functions, then use them to implement the forward step, then implement the backward step, and finally you will use your functions to solve a linear system. **This lab is longer than usual, so make good use of your lab time.**

QUESTION 1. PRINT MATRIX

Download `use_numpy.py` from Quercus and read lines 1-11 to understand how to print lists of lists as matrices by converting lists of lists to NumPy arrays. Write a function `print_matrix(M_lol)` which prints the nested list `M_lol` as a matrix. You will be able to use your function to debug your implementation of Gaussian Elimination.

QUESTION 2. LEADING INDEX

Write a function with the signature `get_lead_ind(row)` which takes in a list of numbers `row`, and returns the index of the first non-zero element of `row`. The function should return `len(row)` if the row contains no non-zero elements.

QUESTION 3. ROW TO SWAP

Write a function with the signature `get_row_to_swap(M, start_i)` which takes in a matrix `M` (represented as a list of lists) and an integer `start_i`, and returns the index of the row that needs to be swapped (permuted/interchanged) with the row `M[start_i]`. The row that needs to be swapped with the row `M[start_i]` is the row at index larger than or equal to `start_i` which has the leading non-zero coefficient that is as far to the left as possible.

This lab has been adapted from one used in previous years by Prof. Michael Guerzhoy.

For example, for

```
M = [[5, 6, 7, 8],
     [0, 0, 0, 1],
     [0, 0, 5, 2],
     [0, 1, 0, 0]]
start_i = 1
```

`[0, 0, 0, 1]` needs to be swapped with `[0, 1, 0, 0]`, so the function should return `3`.

QUESTION 4. ADD ROWS WITH COEFFICIENTS

Write a function with the signature `add_rows_coefs(r1, c1, r2, c2)` which takes in rows `r1` and `r2` (represented as lists of equal lengths) and coefficients `c1` and `c2` (floats), and returns a new list that contains the row `c1*r1 + c2*r2`.

Hint: to create a new list of 10 zeros, you can use `[0]*10`.

QUESTION 5. ELIMINATE

Write a function with the signature `eliminate(M, row_to_sub, best_lead_ind)` which takes in a matrix `M` (represented as a list of lists), `row_to_sub`, the index of the row to subtract from other rows, and `best_lead_ind`, the index of the coefficient to be eliminated in the rows below `M[row_to_sub]`. Assume that `M[row_to_sub]` is all zeros before `M[row_to_sub][best_lead_ind]`.

For example, an input might be

```
M = [[5, 6, 7, 8],
     [0, 0, 1, 1],
     [0, 0, 5, 2],
     [0, 0, 7, 0]]
row_to_sub = 1
best_lead_ind = 2
```

The function should change `M` to become

```
M = [[5, 6, 7,  8],
     [0, 0, 1,  1],
     [0, 0, 0, -3],
     [0, 0, 0, -7]]
```

by adding `-5*M[1]` to `M[2]` and `-7*M[1]` to `M[3]`.

QUESTION 6. FORWARD STEP

Write a function with the signature `forward_step(M)` which takes in an arbitrary matrix `M` (as a list of lists), applies the forward step of Gaussian Elimination to it, and modifies `M` to be the matrix obtained after the forward step is applied. This can be done by repeatedly calling `get_row_to_swap`, swapping rows, and calling `eliminate`. Unlike in ESC103, we

recommend that you keep the entire matrix rather than extracting submatrices. The process of performing the forward step remains essentially the same.

The process is illustrated in Example 1 of the Lab 6 Sample Run document. As you write `forward_step()`, add `print()` statements to `forward_step()` to produce output similar to the sample run (i.e., print out the matrix transformation process, and comments on what is happening at every step).

QUESTION 7. BACKWARD STEP

Write a function with the signature `backward_step(M)` which takes in an arbitrary matrix `M` and applies the backward step of Gaussian Elimination to it. A sample run of the backward step (with automatically-generated explanations of what is going on) is illustrated in Example 2 of the Lab 6 Sample Run document.

QUESTION 8. SOLVING A LINEAR SYSTEM

Now, write a function `solve(M, b)` that solves the equation `Mx = b` for the vector `x`. The idea is to first build the augmented matrix `(M|b)`, then apply Gaussian Elimination to the augmented matrix, and then solve for `x`.

Test your `solve()` function. In `use_numpy.py`, we provide code to quickly perform matrix multiplication in Python, so that you can pick an arbitrary `M` and `x`, obtain `b` by multiplying `M` and `x`, and then verify that your algorithm can recover `x`. For example:

```
M = np.array([[1, -2, 3], [3, 10, 1], [1, 5, 3]])
x = np.array([75, 10, -11])
b = np.matmul(M, x)
x_found = solve(M.tolist(), b.tolist())
print(x_found)   # should recover [75, 10, -11]
```

This lab has been adapted from one used in previous years by Prof. Michael Guerzhoy.